



PROJECT-BASED LEARNING

PHASE-I EVALUATION

**ML-Core: A C++ Implementation of
Fundamental Machine Learning Algorithms**

Team ID: T170

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T170
Team Name: ML Core
Semester: 3rd
Section: ML2,DS1,DS2
Domain: Machine Learning

Team Members

1. Shubhangani Kotiyal – 2510370294
(Team Lead)
2. Ishanaya Agarwal – 2510380249
3. Harshita Joshi – 2510380118

Mentor

Dr. Hriday Kumar Gupta

Interactions completed: 2

Problem Statement & Motivation

Problem Statement

High-level machine learning libraries such as Scikit-learn and TensorFlow let users train models like K-Means or SVM through a single function call, hiding the underlying mathematics – centroid updates, hyperplane computation, gradient descent – behind a black box. This project builds six classical ML algorithms from scratch in C++ to make that mathematics fully transparent.

Motivation

- ▶ High-level libraries abstract away core computations (matrix ops, gradients, distance metrics)
- ▶ Limits understanding of memory

Target Users / Stakeholders

- ▶ CS / ML students learning algorithm internals
- ▶ Self-learners without access to production tooling

Objectives

Project Objectives

1. Implement six classical ML algorithms – Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, and PCA – from scratch in C++.
2. Build a shared utility library for matrix operations, distance metrics, and CSV parsing.
3. Design each algorithm as an independent C++ module, with a separate source file for each algorithm.
4. Validate correctness of each implementation against Python's Scikit-learn.
5. Produce visual output (trend lines, decision boundaries, clusters, dendrograms) for each algorithm.

Key Objective: Deliver six working, from-scratch ML implementations in

Proposed Solution

Proposed Approach

1. Data Layer – CSV parsing & cleaning into numeric arrays
2. Core Utility Layer – matrix operations, distance/gradient/eigen functions
3. Algorithm Module Layer – six independent algorithm implementations
4. Testing – hand-calculated small datasets, then comparison vs.

Scikit-learn

Key Features

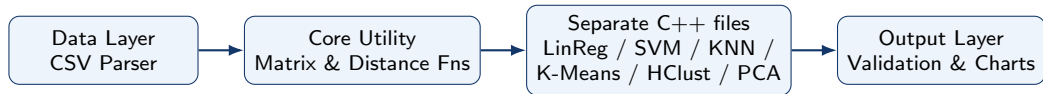
- ▶ Six ML algorithms built from scratch in C++
- ▶ Shared, reusable core utility library
- ▶ Modular architecture – each algorithm independent & testable
- ▶ Correctness verified against Scikit-learn
- ▶ Visual output: trend lines, cluster maps, dendrograms

Integration of PBL Course Concepts

Course	Concepts Applied	Project Module
Data Structures	Vectors and matrices for dataset representation; records for data; searching & sorting for nearest-neighbour selection (KNN); distance-based computations for clustering; tree-like dendrogram construction (Hierarchical Clustering)	Core Utility Layer & Algorithm Modules
OOPs with C++	C++ classes and encapsulation for algorithm state; modular design with one source file per algorithm; reusable functions for shared operations	Algorithm Module Layer design

Requirement: Demonstrate meaningful integration of both subjects in **one** project.

System Architecture / Workflow



Major Components

- ▶ CSV Parser & Data Layer
- ▶ Core Utility Library (matrix ops, distance/math functions)
- ▶ Six Algorithm Modules (Linear Regression, SVM, KNN, K-Means, Hierarchical Clustering, PCA)

Data / Control Flow

- ▶ Raw CSV is parsed & cleaned into numeric data
- ▶ Data is passed to the selected algorithm module via shared utilities
- ▶ Output layer validates results against Scikit-learn and generates charts

Technology Stack & Methodology

Technology Stack

- ▶ Programming: C++ (standard library)
- ▶ Database: None – flat CSV files
- ▶ Tools: Python + Scikit-learn (verification only)
- ▶ IDE: VS Code
- ▶ Version Control: Git / GitHub

Development Methodology

1. Preparation (theory & math study)
2. Core Utility Development
3. Algorithm Implementation
4. Testing & Validation vs. Scikit-learn
5. Finalization & Documentation

No external ML libraries are used; correctness is checked by cross-verifying against Scikit-learn.

Initial Progress & Team Contribution

Progress Achieved

- ▶ Literature & background study of all six algorithms completed
- ▶ Requirements finalized
- ▶ Four-layer modular architecture designed
- ▶ Core utilities & algorithm coding not yet started (planning/design stage)

Individual Contribution

Member	Role	Contribution
Shubhangani Kotiyal	Team Lead	Coordination; Linear Regression & SVM modules
Ishanaya Agarwal	Developer	KNN & K-Means modules
Harshita Joshi	Developer	Hierarchical Clustering & PCA modules

Roadmap, Expected Outcomes & References

Roadmap

1. Phase-I: Proposal & Design
2. Phase-II: Core Utilities & Algorithm Development
3. Phase-III: Testing, Validation & Final Demo

Expected Outcomes

- ▶ Six working, from-scratch ML algorithms in C++
- ▶ Correctness verified against Scikit-learn

References

1. Bishop, *Pattern Recognition & ML* (2006)
2. Hastie et al., *Elements of Stat. Learning* (2009)
3. Cortes & Vapnik, *Support-Vector Networks* (1995)
4. MacQueen, *Classification & Analysis* (1967)
5. Stroustrup, *The C++ Prog. Language*, 4th ed. (2013)
6. Pedregosa et al., *Scikit-learn* (JMLR, 2011)

Thank You
Questions & Suggestions